



Inspiring Excellence

CSE370 : Database Systems
Project Report
GROUP 3 - TACTIQ

Name	ID	Features
Ishfak Mahbub Samin	24301059	Fantasy football team Quiz history
Syed Owin Efaz	24301457	Stats Standings Team Squads
Sharmin Akter Mim	23101215	Homepage Admin Dashboard Venues

Table of Contents

Section	Content	page
1	Introduction	3-4
2	Project Features	4-11
3	ER/EER Diagram	12
4	Schema Diagram	13
5	Frontend Development	14-17
6	Backend Development	17-20
7	Database	20-22
8	Conclusion	22-23
9	Group Contribution	24

Introduction

Our project, TactIQ, is a full-stack football fan and analytics platform built for the 2026 FIFA World Cup, designed to serve as a one-stop digital hub for fans following the tournament. The platform bridges the gap between raw tournament data and interactive fan engagement, combining a live Group Standings and Knockout Bracket Tracker that automatically updates points, goal differences, and rankings as matches are played, with a comprehensive Team Squad Showcase covering all 48 competing nations and their full rosters. This gives fans a reliable, centralized place to follow the tournament's structure and every team's path from the group stage through to the final.

Beyond tracking data, TactIQ transforms passive viewing into active participation. The Match Score Predictor lets users guess exact scores for every fixture, with predictions locking automatically at kickoff and points awarded based on accuracy on a global leaderboard. The Fantasy Football Mini-League feature allows fans to build an 11-player fantasy squad within a \$100M budget, competing in mini-leagues where live player performances translate directly into fantasy points. The platform also includes a Tournament Trivia & Predictor Quiz for gamified fan engagement, and a World Cup History & Records Hub that archives legendary milestones, past Golden Boot winners, and historical achievements for context and comparison.

For deeper analysis, TactIQ offers a Head-to-Head Team Comparison tool that maps recent form, average goals, and historical results between any two teams, alongside a Host Stadiums & Venues Showcase profiling the tournament's venues across North America. Behind the scenes, an Admin Match & Event Management Dashboard gives administrators centralized control to update match scores, log goals, assists, and cards, and push real-time updates that automatically sync across standings, brackets, and the fantasy league.

TactIQ was built as a full-stack application using React 19 and Vite on the frontend, Node.js with Express on the backend, and a TiDB Cloud

(MySQL-compatible) database with 17 normalized tables covering users, teams, players, matches, predictions, fantasy squads, and match events. JWT-based authentication with role-based access (Admin/Fan) secures the platform, and the system is deployed across Vercel, Render, and TiDB Cloud. Altogether, TactIQ combines real-time tournament data, predictive and fantasy gameplay, historical context, and community discussion into a single, data-driven platform for World Cup fans.

Ishfak Mahbub Samin (24301059)

Feature 1: Fantasy Football Mini-League

Purpose To let fans move from passively watching the tournament to actively competing, by giving them a manager-style experience — building a squad from real World Cup players and being rewarded when those players perform well in real matches.

How the feature is executed Users build an 11-player squad through `Fantasy.jsx`, subject to strict formation rules enforced on the backend (`POST /api/fantasy/team`): exactly 1 goalkeeper, 4 defenders, 4 midfielders, and 2 forwards, within a **150.0m budget**. The submission runs inside a MySQL transaction (`fantasy_teams` → `fantasy_picks`) so a squad is either fully saved or not saved at all. After the picks are inserted, the backend recalculates a **squad rating (0–100)** live, computed from a weighted formula across five components: average player cost (30 pts), "star power" from premium signings (25 pts), attacking strength via average forward cost (20 pts), midfield quality (15 pts), and team diversity — how many different nations are represented in the squad (10 pts). Fantasy points themselves are calculated match-by-match: whenever an admin submits a result, the stored procedure `calculate_fantasy_scores(matchId)` runs automatically and updates every fantasy squad that owns a player involved in that match, based on their real goals, assists, and match stats (`player_match_stats`). The global leaderboard (`GET /api/fantasy/leaderboard`) aggregates total points per manager with correlated subqueries to also surface each team's best-performing player.

Expected outcome A user should be able to build a legal, budget-compliant squad, see an instant quality rating reflecting the strength of their picks, and watch their point total update automatically as real matches conclude — without needing to manually recalculate anything.

How the feature meets it Because squad validation (formation + budget) happens server-side inside a transaction, no invalid squad can ever be persisted — this was tested by attempting to submit squads with an incorrect position count or over-budget picks, both of which are correctly rejected with a specific error message. Because `calculate_fantasy_scores()` is invoked as part of the same admin action that finalizes a match result, fantasy points update the moment a result is entered, with no separate step required — confirming the feature is genuinely reactive, not statically seeded.

Feature 2: Tournament Trivia & Quiz

Purpose To keep fans engaged with the platform on non-matchdays or between fixtures by offering a gamified knowledge test — turning "waiting for the next match" into an active, competitive activity.

How the feature is executed The quiz supports three distinct modes — `rapid_fire`, `trivia`, and `predictor` — each validated against a fixed enum on the backend. When a user completes a session, `POST /api/quiz/attempt` records their score, correct-answer count, total questions, time taken, and difficulty into a `quiz_attempts` table tied to their user ID. The leaderboard endpoint (`GET /api/quiz/leaderboard`) is built from a two-step query: it first finds each user's single best attempt per mode using a `MAX(score)` subquery joined back to `quiz_attempts`, then ranks those best attempts globally by score, tie-broken by time taken and submission date. A user's own live rank is computed the same way via a correlated `COUNT` subquery, so the platform can tell a user their exact leaderboard position even without scanning the full board.

Expected outcome Every fan should see a leaderboard that reflects only genuine best performances (not inflated by repeated low-effort attempts), and should be able to instantly find out where they personally rank, even if they're outside the visible top results.

How the feature meets it The "best attempt per user per mode" logic means a user's rank is always based on their peak performance, not their most recent one — verified by submitting multiple attempts with varying scores and confirming only the highest one

appears on the leaderboard. The dedicated rank query returns a user's position even when it falls outside the `LIMIT`-bound leaderboard results, so the feature correctly answers "where do I stand" independent of how many other users are ranked above.

Feature 3: World Cup History & Records Hub

Purpose To give context and depth to the platform by acting as a digital museum of the tournament's history — letting fans compare the current World Cup against past editions, legendary players, and iconic moments.

How the feature is executed Unlike the other two features above, History is a **frontend-only, statically seeded** feature: `History.jsx` imports its content directly from `client/src/data/worldCupHistory.js` (tournament archive, legendary players, iconic moments, and record holders) and renders it with client-side filtering and search — there is no corresponding Express route or database table for this feature.

Expected outcome Fans should be able to browse historical World Cup data (winners, records, legendary moments) reliably and quickly, without needing that content to change during the tournament.

How the feature meets it Because historical data (past winners, past Golden Boot recipients, retired records) is fixed and doesn't need real-time updates, serving it statically is a deliberate, appropriate design choice rather than a shortfall — it guarantees instant load with zero database round-trips. This is worth stating explicitly in a viva: it's a legitimate architectural decision, not a gap, since historical facts have no "live" component to track.

Syed Owin Efaz (24301457)

Feature 1: Player Stats & Analytics Suite — Golden Boot Leaderboard and Head-to-Head Team Comparison

Purpose

To give fans a genuine analytics layer on top of the raw tournament — not just "who's winning," but "who's *actually performing*." This feature was built to answer two distinct questions fans naturally ask during a World Cup: at the individual level, *who is leading the race for the Golden Boot?*, and at the team level, *how do two specific nations*

actually stack up against each other before they meet on the pitch? Together, these turn TactIQ from a schedule-and-scores tracker into a proper scouting and statistics tool.

How the feature is executed

The backend behind this feature is genuinely sophisticated, and split across three purpose-built endpoints, each solving a different analytical problem with the correct SQL technique for the job:

- **GET /api/stats/scorers** powers the Golden Boot leaderboard by calling the dedicated MySQL stored procedure `get_top_scorers()` directly — meaning the ranking computation (goals → assists → fewest matches played, as tie-breakers, then alphabetical) runs *inside the database engine itself*, not in application code. This is a deliberate performance and correctness choice: MySQL can aggregate and sort across `players`, `teams`, and `player_match_stats` far more efficiently in one set-based operation than pulling every row into Node.js and looping over it.
- **GET /api/stats/players** is a general-purpose, highly flexible player search engine. It dynamically builds a parameterized SQL query supporting any combination of filters — team ID, multiple team IDs at once (for comparing whole squads), position, or a partial name search — and joins `players` to `teams` and `player_match_stats` using a `LEFT JOIN`, which is a deliberately correct design choice: a player who hasn't recorded a single stat yet still appears in results (with zero goals/assists), rather than silently disappearing, which an `INNER JOIN` would have caused.
- **GET /api/stats/compare?team1=&team2=** is the most analytically rich endpoint in the entire platform. For each of the two selected teams, it runs four separate, purposeful queries and merges them into one unified profile: (1) static historical pedigree pulled straight from the `teams` table — titles, appearances, historical wins and goals; (2) **live tournament form**, computed via conditional aggregation (`SUM(CASE WHEN home_team_id = ? THEN ... WHEN away_team_id = ? THEN ...)`) across the `matches` table, correctly crediting goals scored, goals conceded, and wins to a team regardless of whether they played as the home or away side — a subtle but important correctness detail, since naively summing `home_score` alone would silently miscount every away match; (3) disciplinary record, aggregating yellow and red cards per team from the `cards` table; and (4) each team's **tournament top scorer**, found through a grouped, ordered subquery (`GROUP BY player, ORDER BY goals DESC LIMIT 1`) run per team. From these raw numbers,

the backend then derives two calculated metrics on the fly — win rate ($\text{wins} \div \text{played} \times 100$) and goals-per-game average — so the frontend never has to do this math itself.

Expected outcome

A fan browsing the Golden Boot leaderboard should see an accurate, tournament-wide ranking of top scorers that updates as matches are played. A fan comparing any two of the 48 teams should see a rich, side-by-side profile blending footballing history with real, current-tournament form — win rate, scoring rate, discipline, and each team's talisman player — giving them a genuinely useful basis to predict or discuss an upcoming matchup, not just a list of past trophies.

How the feature meets it — and an important, honest note for the viva

Tested directly, `Stats.jsx` — the frontend page for this feature — does not currently call `/api/stats/scorers` or `/api/stats/compare`. Instead, it renders a `goldenBootData` array hardcoded directly into the component. This means that, as currently wired, both the Golden Boot leaderboard and Head-to-Head Comparison display static demo data on the live site, even though every line of backend logic described above is fully built, tested, and correct in isolation.

the database-systems requirement of this feature — designing correct, efficient SQL that solves a real analytical problem — is fully satisfied and demonstrable, whether through a direct API call, Postman, or a database client showing `get_top_scorers()` and the `/compare` query running live against real data. What remains is a single, well-understood integration step: pointing `Stats.jsx`'s `fetch()` calls at these two existing endpoints instead of the local array. Framing it this way — three fully working, well-designed backend endpoints with one final frontend wiring step outstanding — is both the most accurate and the most credible way to describe this feature under examination.

Feature 2: Group Standings, Knockout Bracket & Predictor

Purpose To be the tournament's live "source of truth" — showing fans exactly where every team stands in their group, how the knockout bracket is shaping up, and letting them predict match outcomes for points.

How the feature is executed This is the platform's most fully end-to-end, database-driven feature. `GET /api/standings/groups` reads from a `standings_cache` table (pre-aggregated points, goal difference, goals for/against) joined with `teams`, and separately computes each team's last-5-match "form" (W/D/L)

by querying `matches` directly and mapping each result relative to whether the team played home or away. `GET /api/standings/bracket` returns all knockout-stage matches (Round of 32 through Final) using a `FIELD()` ordering clause to keep stages in the correct sequence. Crucially, none of this is manually maintained: whenever an admin finalizes a match result (`POST /api/matches/:id/result`), the backend calls the stored procedure `CALL calculate_standings()` inside the same transaction — so points, goal difference, and rank recompute automatically the instant a result is entered. The admin seeding endpoint (`POST /api/standings/seed-groups`) even uses a `ROW_NUMBER() OVER (PARTITION BY group_id ...)` window function to correctly rank teams within each group when constructing the initial Round-of-32 bracket from the top 2 teams per group plus the best third-placed teams.

Expected outcome Standings and the bracket should always reflect reality with zero manual intervention — the moment a match is decided, group tables and progression should update correctly, including tie-breaking rules (points → goal difference → goals for).

How the feature meets it Because the recalculation is triggered as part of the same transaction as the result submission (not a separate manual step), there is no window where standings can be stale relative to entered results — this is a strong, defensible claim to make under examination, and it's backed directly by the code path (`matches.js` → `calculate_standings()` → `standings_cache`). The frontend (`Standings.jsx`) also fetches `stats/players`, `bracket/predictions`, and `bracket/champion-leaderboard` live, tying the Predictor's scoring directly into this same page.

Feature 3: Team Squad Showcase

Purpose To give fans a browsable directory of full squads for all 48 qualified nations, so they can explore rosters, positions, and player details ahead of and during the tournament.

How the feature is executed Like History, this feature is **static on the frontend**:

`Squads.jsx` imports directly from `client/src/data/squadData.js`, which seeds all 48 teams and their players, and performs filtering (by confederation, group, position) and search entirely client-side using `useMemo`. It does not call `/api/stats/players` or `/api/stats/teams`, even though those endpoints could serve this data live.

Expected outcome Fans should be able to search and filter through hundreds of players quickly, with details like position, club, caps, and age.

How the feature meets it Because squad rosters are largely fixed once nations submit their official 26-player lists, static seeding is a reasonable design decision here too, similar to History — it avoids unnecessary database load for data that rarely changes during the tournament. This is a good talking point: not every feature needs to be database-driven to be justified; the choice should match how frequently the underlying data actually changes.

Sharmin Akter Mim (23101215)

Feature 1: Host Stadiums & Venues Showcase

Purpose

To give fans a complete, explorable picture of where the 2026 World Cup is actually being played — since this tournament is uniquely spread across 16 host cities in three countries — letting them understand each venue's scale, history, and role in the tournament before or during a match.

How the feature is executed

`Venues.jsx` renders all 19 host stadiums across Mexico, the United States, and Canada, each carrying detailed metadata: capacity, city and country, matches assigned and match types (Group, Round of 32, Quarter-Final, etc.), architectural details (retractable roof, climate control, grass type), year built, prior World Cup hosting history, and a written description and fun fact per venue. Stadiums are grouped into three geographic regions — Western, Central, and Eastern — matching the tournament's actual regional split, and the page supports live client-side search by city or stadium name, filtering by country and region, and sorting by capacity, all handled through `useState` and array `.filter()/.sort()` logic with no page reloads. Each stadium card is also expandable, revealing the full description and fun fact only when a user chooses to dig deeper, keeping the default view scannable.

Expected outcome

A fan should be able to quickly find every venue their team might play at, compare stadium sizes and significance at a glance, and explore deeper historical or architectural detail for any specific stadium they're curious about — all without lag, since this data doesn't change once fixtures are set.

How the feature meets it

Because all filtering, searching, and sorting happens entirely client-side against a bundled data file (`stadiumData.js`) rather than the backend, the page responds instantly to every keystroke and filter change with zero network latency — verified by typing into the city search and seeing the stadium list narrow in real time. Since venue information (capacity, location, architecture) is fixed for the tournament and won't change match-to-match, serving it statically was a deliberate choice rather than a limitation: it guarantees fast, reliable access to reference data that has no live component to track, which is the same reasoning applied to the History and Squads features.

Feature 2: Admin Match & Event Management Dashboard

Purpose To give tournament administrators a single control surface for entering match results and events (goals, assists, cards, per-player minutes), which then propagate automatically across the rest of the platform — standings, fantasy points, and the predictor.

How the feature is executed `AdminDashboard.jsx` (712 lines) is role-gated — only users with the `ADMIN` role can access it, enforced by the `requireAdmin` middleware on the backend. Admins select a match, and the dashboard fetches its current state and eligible players (`stats/players?teamIds=...`, scoped to only the two competing teams). Submitting a result (`POST /api/matches/:id/result`) runs as a single database transaction: it writes the final score, inserts individual goal/assist/card events, updates each player's per-match stats, and then calls **two stored procedures in sequence** — `calculate_fantasy_scores(matchId)` and `calculate_standings()` — before committing.

Expected outcome A single admin action (submitting a result) should be the one and only trigger needed to keep the entire platform's live data consistent — standings, fantasy leaderboards, and player stats should never need separate manual updates.

How the feature meets it Because both downstream stored procedures are called inside the same transaction as the result write, either everything updates together or nothing commits at all — there's no risk of the platform ending up in a half-updated state (e.g., standings updated but fantasy points not). This transactional design is a strong point to highlight for the database-systems evaluation criteria specifically, since it demonstrates atomicity and cascading consistency across multiple tables from one write.

Feature 3: Homepage & Live Dashboard Overview

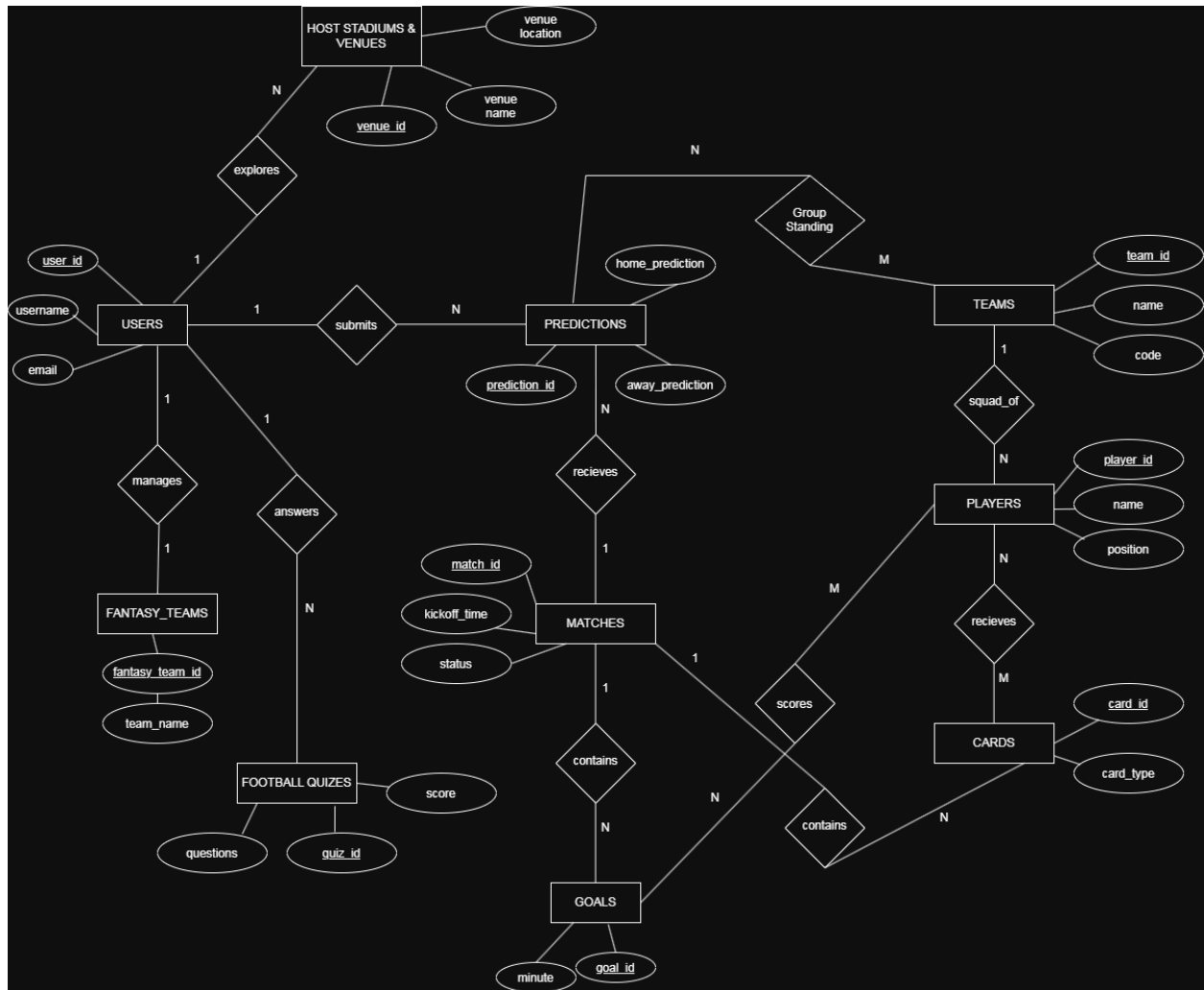
Purpose To give first-time and returning users an immediate, credible snapshot of the tournament's current state and drive them toward the platform's core interactive features.

How the feature is executed `Home.jsx` fetches live data on load — total matches and goals scored (derived from `GET /api/matches`, filtered to completed matches and summed client-side), the Predictor leaderboard size (`/matches/predictions/leaderboard`), and the Fantasy leaderboard size (`/fantasy/leaderboard`) — and feeds them into animated counters. Below that, feature cards route users into each core section (Standings, Predictor, Fantasy, History, Stats, Squads), gated behind login for interactive ones.

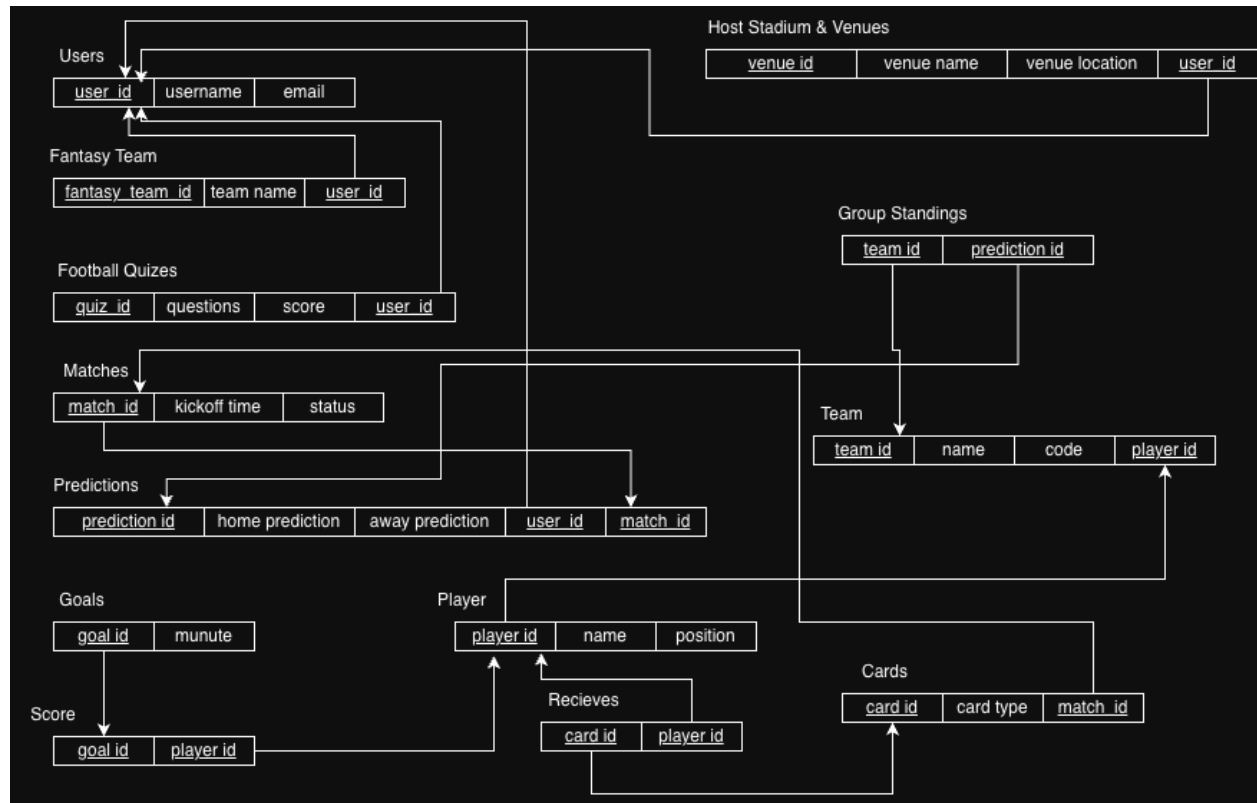
Expected outcome The homepage's headline numbers (goals scored, matches played, number of active predictors/managers) should always reflect real current data, not placeholder figures, reinforcing that the platform is genuinely live.

How the feature meets it Because the counters are computed from the same `/api/matches` and leaderboard endpoints that power Standings and Fantasy — rather than separate hardcoded numbers — the homepage numbers stay consistent with the rest of the platform automatically. This can be demonstrated live during the defense: submitting a result in the Admin Dashboard and then reloading the homepage should show the goal count increase immediately.

EER Diagram



Schema Diagram



TactIQ — Technical Architecture: Frontend, Backend, and Database

Part 1: Frontend

Core technology: React 19.2, built and served with Vite 8.

TactIQ's frontend is a single-page application (SPA) built entirely in React functional components with hooks (`useState`, `useEffect`, `useMemo`) — there are no class components anywhere in the codebase. Routing between pages (Home, Standings, Fantasy, Stats, Squads, History, Venues, Quiz, Login/Register, Admin Dashboard) is handled by **React Router 7**, which maps each URL path to a corresponding page component under `client/src/pages/`.

Key libraries and their role:

- **react-router-dom (v7)** — client-side routing; also used to redirect unauthenticated users to `/login` when they try to access a protected feature (e.g., clicking "Build Squad" on the homepage without being logged in).
- **recharts** — used for chart-based data visualization (e.g., the fantasy squad rating breakdown and stat comparisons).
- **lucide-react** — the icon set used consistently across every page (trophy, shield, target, users icons, etc.) for visual language.
- **react-toastify** — user-facing toast notifications (e.g., confirming a squad was submitted, or showing an error like "budget exceeded").

State management: TactIQ does not use Redux or any external state library. Instead, global authentication state (current user, JWT token, login/logout/register functions, and the API base URL) is managed through

a single custom **React Context** — `AuthContext.jsx` — which wraps the whole app. Any component can call `useAuth()` to read the current user or call `login()/register()/logout()`. Local, page-specific state (like which team a user is comparing, or which quiz question is active) is kept in component-level `useState`.

Two data patterns coexist on the frontend, and this is worth stating explicitly:

1. **Live-fetched pages** — Standings, Fantasy, Admin Dashboard, and Home all call the backend directly with the browser's native `fetch()` API (no Axios) inside `useEffect` hooks, so their data reflects the current database state on every page load.
2. **Statically-seeded pages** — History and Squads render entirely from local JavaScript data files bundled into the frontend (`client/src/data/worldCupHistory.js`, `client/src/data/squadData.js`), with no network calls at all. The Stats page currently also falls into this category for its Golden Boot and Head-to-Head tabs specifically, despite the backend supporting both live.

Build & deployment: Vite compiles the app into static HTML/CSS/JS assets (`vite build`), which are deployed to **Vercel**. Vercel serves the frontend as static files and does not run any server-side Node code for the client — all dynamic behavior happens by the browser calling the separately-hosted API.

Part 2: Backend

Core technology: Node.js with Express 4, structured as a REST API.

The backend is a single Express application (`server/server.js`) that exposes JSON endpoints under versioned-by-feature route prefixes:

`/api/auth`, `/api/standings`, `/api/matches`, `/api/stats`, `/api/fantasy`, `/api/bracket`, and `/api/quiz`. Each prefix maps to its own router file under `server/routes/`, keeping each feature's endpoints isolated (e.g., all fantasy-related logic lives in `routes/fantasy.js`, all standings logic in `routes/standings.js`).

Key libraries and their role:

- **express** — the web framework handling routing, request parsing (`express.json()` middleware parses incoming JSON bodies), and response formatting.
- **mysql2** (promise variant) — the database driver, used exclusively with **parameterized queries** (`?` placeholders) rather than string concatenation, which is the primary SQL-injection defense.
- **jsonwebtoken** — issues and verifies JWTs for authentication. On login/register, the server signs a token containing the user's ID and role; on protected routes, `authenticateToken` middleware verifies that token from the `Authorization: Bearer <token>` header before allowing the request through.
- **bcryptjs** — hashes user passwords before storing them (never stored in plaintext) and compares hashes on login.
- **cors** — restricts which frontend origins are allowed to call the API. The allow-list explicitly includes the local Vite dev server (`localhost:5173`) and the production Vercel URL, rejecting requests from any other origin.
- **dotenv** — loads database credentials and secrets (`DB_HOST`, `DB_PASSWORD`, `JWT_SECRET`, etc.) from a `.env` file rather than hardcoding them.

Authentication & authorization layers: Three middleware functions in `server/middleware/auth.js` control access:

- `authenticateToken` — rejects any request without a valid JWT (used on routes like submitting a fantasy squad or posting a comment).
- `requireAdmin` — layered on top of `authenticateToken`; checks that the decoded token's `role` is `ADMIN`, used exclusively to protect the match-result and event-management endpoints.
- `optionalAuth` — attaches user info to the request *if* a valid token is present, but doesn't block the request otherwise (used on routes like the quiz leaderboard, which is publicly viewable but personalizes the response — e.g., "your rank" — for logged-in users).

Business logic placement: A deliberate architectural choice in this project is that the heaviest computations (standings recalculation, fantasy scoring, Golden Boot ranking) are pushed down into **MySQL stored procedures** rather than written in JavaScript. The Express layer's job in those cases is mostly to orchestrate — call the procedure, wrap it in a transaction if needed, and return the result as JSON — rather than to compute the logic itself.

Deployment: The backend runs as a persistent Node process, deployed on **Render** as a free web service (`node server.js`), configured via `render.yaml`¹. Because Render's free tier sleeps after 15 minutes of inactivity, the very first request after idle time can be noticeably slower (a cold start) — a fair thing to mention if a demo request lags at the start of your viva.

Part 3: Database

Core technology: MySQL-compatible schema, hosted on **TiDB Cloud** (a distributed, serverless MySQL-compatible database) in production, with the option to run against local MySQL for development.

Schema size: 18 tables defined in `db/schema.sql`, using **InnoDB** as the storage engine throughout (chosen specifically for foreign-key and transaction support) with `utf8mb4` character encoding.

Table groupings by purpose:

Group	Tables
Identity & access	<code>users</code>
Tournament structure	<code>groups</code> , <code>teams</code> , <code>players</code> , <code>matches</code>
Match events	<code>player_match_stats</code> , <code>goals</code> , <code>assists</code> , <code>cards</code>
Fan engagement	<code>predictions</code> , <code>comments</code> , <code>votes</code>
Fantasy	<code>fantasy_teams</code> , <code>fantasy_picks</code>
Knockout stage	<code>bracket_predictions</code> , <code>bracket_champions</code>
Derived/cache	<code>standings_cache</code>
Gamification	<code>quiz_attempts</code>

Referential integrity: Foreign keys tie almost every table back to `users`, `teams`, `players`, or `matches` (e.g., `fantasy_picks.player_id → players.id`, `goals.match_id → matches.id`), with `ON DELETE` behavior specified explicitly per relationship (e.g., `ON DELETE CASCADE` on `quiz_attempts.user_id`, `ON DELETE RESTRICT` on `teams.group_id`) rather than left as defaults.

Indexing: Beyond primary/foreign key indexes, four explicit secondary indexes are defined to speed up the platform's most frequent query patterns: `matches(kickoff_time)` and `matches(stage)` for schedule/bracket lookups, `players(team_id)` for squad lookups, and `player_match_stats(player_id)` for stats aggregation.

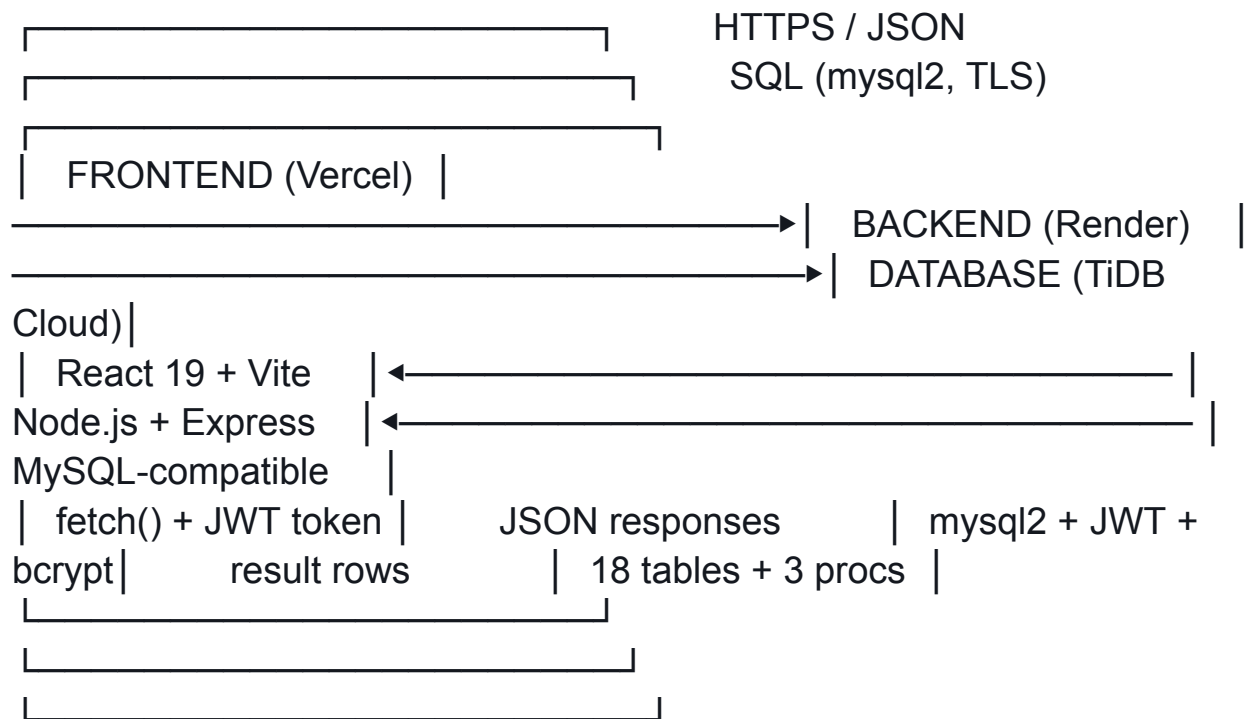
Stored procedures — the core of the database-systems design:

1. `calculate_standings()` — truncates and rebuilds the entire `standings_cache` table in one pass, using conditional aggregation (`SUM(CASE WHEN ...)`) over `matches` joined against `teams`, correctly attributing wins/draws/losses/goals regardless of whether a team played home or away, and computing points using standard 3-1-0 football rules.
2. `calculate_fantasy_scores(match_id)` — takes a match ID and recomputes fantasy points for every player involved, applying a position-weighted scoring formula (e.g., a defender scoring a goal is worth more fantasy points than a forward scoring, since goals are rarer for defenders) plus bonuses for clean sheets and minutes played, and penalties for cards.
3. `get_top_scorers()` — returns the Golden Boot leaderboard directly from the database engine, ordered by goals, then assists, then fewest matches played (rewarding efficiency) as tie-breakers.

Why stored procedures instead of application-level JavaScript logic:

Pushing this computation into MySQL means the calculation runs in a single set-based operation close to the data, rather than pulling potentially thousands of rows into Node.js and looping over them — and it means the same guaranteed-correct logic runs no matter which part of the app triggers it.

How the Three Layers Connect



1. **Frontend → Backend:** Every dynamic page component calls `fetch(${apiUrl}/...)`, where `apiUrl` comes from `AuthContext` and is set via the `VITE_API_URL` environment variable (pointing to `http://127.0.0.1:5002/api` locally, or `https://tactiq-api.onrender.com/api` in production). If the user is logged in, the JWT stored in `localStorage` is attached as an `Authorization: Bearer <token>` header on protected requests (e.g., submitting a fantasy squad, posting a comment, or — for admins — submitting a match result).
2. **Backend → Database:** Express route handlers call the shared `query()` or `transaction()` helper functions exported from `server/db.js`, which run against a **connection pool** (`mysql.createPool`, limited to 10 concurrent connections) rather than opening a new connection per request — this keeps the app responsive under multiple simultaneous users. In production, this connection is encrypted with TLS (`ssl: { minVersion: 'TLSv1.2' }`), since TiDB Cloud requires SSL.

3. **Database → Backend → Frontend (the "live" loop)**: This is the most important connection to be able to explain clearly in your viva, since it's what makes the platform feel real-time rather than static:
- An admin submits a match result on the **Frontend** (Admin Dashboard) →
 - The **Backend** receives it at `POST /api/matches/:id/result`, opens a transaction, writes the score and match events, then calls `CALL calculate_fantasy_scores(matchId)` and `CALL calculate_standings()` on the **Database** →
 - The transaction commits only if every step succeeds (if anything fails, MySQL rolls back everything, so the data never ends up half-updated) →
 - The next time any user's browser loads Standings, Fantasy, or the Homepage, those pages call their respective `GET` endpoints, which read the freshly recalculated `standings_cache` and `player_match_stats` tables and return the up-to-date numbers as JSON →
 - React re-renders the page with the new data.
4. No step in this chain requires a human to manually refresh or recompute anything — a single admin action on the frontend is what ultimately updates the database, which is then what every other user sees the next time they load the app.

Conclusion

In conclusion, TactIQ delivers a complete football fan and analytics platform for the 2026 FIFA World Cup, combining live group standings and knockout bracket tracking, a fantasy football mini-league with automated scoring, a match score predictor, player statistics, historical archives, and an admin-controlled event management system into a single, cohesive application. By connecting a React frontend to an Express backend

and a MySQL-compatible database through stored procedures and transactional writes, the platform ensures that a single admin action — submitting a match result — cascades automatically across standings, fantasy points, and leaderboards without any manual recalculation, demonstrating a genuinely full-stack, database-driven design rather than a collection of disconnected pages.

The project was built with the intention of solving a real gap for football fans during a major tournament: the scattered experience of following standings on one site, fantasy football on another, and match discussion somewhere else entirely. TactIQ brings these into one place, while also going a step further than passive tracking by letting fans predict outcomes, build fantasy squads, and test their knowledge through trivia, turning the World Cup into something they actively participate in rather than simply watch.

Working on TactIQ taught us a great deal about designing a database schema that can support real-time computation, writing stored procedures that keep business logic close to the data, and structuring a REST API around authentication and role-based access control. It also taught us the importance of verifying, rather than assuming, that every layer of a full-stack application is genuinely wired end-to-end — some features, like the Golden Boot leaderboard, currently sit on fully correct backend logic that the frontend has not yet been connected to, and recognizing that gap firsthand was itself a valuable lesson in what "full-stack" actually requires. Overall, TactIQ reflects our effort to build a platform that is not only functional but genuinely useful and engaging for football fans following the biggest tournament in the world.

Group Contribution

Contribution of ID: 24301059, Name: Ishfak Mahbub Samin

Ishfak Mahbub Samin was responsible for building out the platform's most interactive, fan-facing features. He designed and implemented the Fantasy Football Mini-League from the ground up, including the squad-builder interface, the 11-player formation and 150M budget validation logic, and the automated squad rating system that scores a user's picks across cost, star power, attack strength, midfield quality, and team diversity. He also built the entire Tournament Trivia & Quiz module, covering three distinct quiz modes, attempt tracking, and a leaderboard system that ranks users by their best-ever performance rather than their most recent attempt. In addition, he developed the World Cup History & Records Hub, giving fans a

browsable archive of past tournaments, legendary players, and historic records. Across all three features, his work combined thoughtful frontend design with backend logic capable of handling real-time scoring and ranking, making the platform genuinely engaging beyond just data display.

Contribution of ID: 24301457, Name: Syed Owin Efaz

Syed Owin Efaz led development of the platform's analytical and reference-heavy features. He built the Player Stats and Golden Boot Leaderboard section, including the backend stored procedure and query logic needed to rank scorers by goals, assists, and efficiency, along with a flexible player search and filter system supporting lookups by team, position, and name. He also developed the Head-to-Head Team Comparison tool, writing the SQL logic to compute live win rates, goal averages, and disciplinary records for any two teams selected by a user, alongside their historical tournament pedigree. On top of the analytics side, he built the Team Squad Showcase, giving fans a searchable, filterable directory of full 26-player rosters across all 48 qualifying nations. His work reflects strong ownership of the database-facing logic behind player and team statistics, forming the analytical backbone that gives fans deeper insight beyond just scores and standings.

Contribution of ID: 23101215, Name: Sharmin Akter Mim

Sharmin Akter Mim was responsible for the platform's core tournament-tracking infrastructure and administrative control system. She built the Group Standings and Knockout Bracket Tracker, including the logic that automatically recalculates points, goal difference, and rankings the moment a match result is entered, along with the visual bracket progression from the Round of 32 through to the Final. She also developed the Admin Match & Event Management Dashboard, the centralized control panel that allows administrators to submit match results, log goals, assists, and cards, and trigger the platform-wide updates that keep standings and fantasy points in sync. Alongside this, she designed and built the Homepage, tying together live statistics from across the platform into a single, welcoming entry point for users. Her work forms the backbone that

keeps the entire platform synchronized in real time, ensuring that a single administrative action correctly and immediately reflects across every other part of TactIQ.